Trabalho 01

Análise de complexidade em estruturas de listas

Gabriel Leite Fonseca e César Augusto Bispo Santos

30/03/2026

Seção 01

Responsabilidades atividades

Atividade	Membro Responsável
Criação classe ListaEncadeada	Gabriel Leite Fonseca
Criação classe ListaEncadeada	César Augusto Bispo Santos
Implementação Programa que use Biblioteca	César Augusto Bispo Santos

Github

github: https://github.com/Gabedf/TPA/tree/main/Listas/src

Seção 02

2 - Análise Matemática de complexidade dos Algoritmos

2.1 - Quantidade de Nós

```
public int quantidadeNos() {
    return this.qtd;
}
```

A forma optada por se implementar o método responsável por retornar a quantidade de nós foi a mais simples, tendo um atributo de quantos nós existe dentro da classe e apenas retornando esse atributo. O método é **O(1)** por ser constante, independente da quantidade de nós sempre retornando a mesma no mesmo tempo.

2.2 - Adicionar

```
public void adicionar(T novoValor) {
    Node<T> novo_ultimo  = new Node<T>();
    novo_ultimo.value = novoValor;

    if (this.first == null) { this.first = this.last = novo_ultimo; }
    else {
        this.last.next    = novo_ultimo;
        this.last         = novo_ultimo;
        this.last.next    = null;
    }
    this.qtd += 1;
}
```

A complexidade do método **adicionar** é **O(1)**, pois é realizada uma inserção diretamente no final da lista, independente de quantos elemento a lista possua e sem precisar percorre-los um a um. Isso é possível pela referência apontada por *last* e pela natureza de adicionar sem se preocupar com a posição em que o elemento será inserido.

A título de comparação, em um método de inserção no qual o elemento deve ser adicionado em uma posição específica definida, por exemplo, por um índice informado pelo usuário, a complexidade de tempo passa a ser O(n).

Isso ocorre porque é necessário percorrer a estrutura elemento por elemento até alcançar a posição desejada. Esse processo de busca é linear em relação ao número de elementos da lista. Após localizar a posição correta, as demais operações de inserção possuem custo constante, mas não alteram a complexidade total do método, que permanece O(n) devido ao custo dominante.

2.3 - Pesquisar

```java
    public T pesquisar(T valor) {
        No<T> noAtual = this.primeiro; C1

        while (noAtual != null) {
            if (Objects.equals(noAtual.getValor(), valor)) {
                return noAtual.getValor();
            }
            noAtual = noAtual.getProx();
        }
        return null; C3
    }
```

N VEZES

C2

Figure 1: Código método de pesquisa

> **Note**
>
> Para facilitar a análise, as linhas do código foram agrupadas em constantes (C1, C2, C3), representando operações de custo fixo (abordagem adotada para restante do relatório). O ponto principal está no laço de repetição, que pode executar até n vezes, dependendo do tamanho da lista. Por isso, o comportamento do algoritmo é determinado por esse trecho.

A complexidade do método pesquisar é **O(n)**, pois a busca é realizada de forma sequencial ao longo da lista encadeada. Para localizar um valor, o algoritmo percorre os nós um a um, comparando seus conteúdos até encontrar o elemento desejado ou atingir o final da estrutura.

No pior caso, quando o elemento não está presente (ou encontra-se na última posição), o método percorre todos N elementos da lista até que o ponteiro atual se torne nulo, encerrando a busca. Nesse cenário, o número de operações cresce proporcionalmente ao tamanho da lista, caracterizando um crescimento linear.

2.4 - Remover

```
53      public boolean remover(T valor) {
54          No<T> noAtual    = this.primeiro; C1
55          No<T> noAnterior = null; C2
56
57          while (noAtual != null) {
58              if (Objects.equals(noAtual.getValor(), valor)) {
59                  if (noAnterior == null) {
60                      // remoção do primeiro nó
61                      this.primeiro = noAtual.getProx();
62                      if (this.primeiro == null) { this.ultimo = null; }
63                  } else if (noAtual.getProx() != null) {
64                      // remoção de nó do meio
65                      noAnterior.setProx(noAtual.getProx());
66                  } else {
67                      // remoção do último nó
68                      this.ultimo = noAnterior;
69                      noAnterior.setProx(null);
70                  }
71                  noAtual.setProx(null); // limpa referência do nó removido
72                  this.qtd--;
73                  return true;
74              }
75
76              noAnterior = noAtual;
77              noAtual    = noAtual.getProx();
78          }
79          return false; C4
80      }
81  }
```

C3

Figure 2: Código método de pesquisa

O método remover, assim como o método pesquisar, apresenta complexidade de tempo **O(n)**, pois depende de uma percurso sequencial da lista para localizar o elemento a ser removido.

Embora o corpo do laço contenha mais operações em comparação ao método de busca, essas operações possuem custo constante. Mesmo que existam diferenças de tempo entre elas, tais variações são desprezíveis na análise, podendo ser agrupadas em uma constante (C3).

No pior caso, quando o elemento não está presente ou encontra-se na última posição, o algoritmo percorre todos os **N** elementos da lista. Assim, o custo total pode ser representado como

proporcional a **N** e caracterizar um crescimento linear.

Dessa forma, a complexidade do método é **O(n)**.

Seção 03

3.1 - Implementação

Para avaliar o desempenho prático da nossa implementação de estruturas de dados, desenvolvemos um módulo gerador de arquivos e um mecanismo de benchmark.

Os arquivos de entrada foram gerados utilizando a classe `GeradorDados`. Este módulo cria registros de alunos contendo uma matrícula (de forma sequencial, para garantir unicidade), um nome aleatório e uma nota. Todos os dados são gravados em arquivos `.txt` separados por ponto e vírgula (`;`). Abaixo, um trecho de código que consolida a geração:

```
// Trecho do arquivo GeradorDados.java
Random random = new Random();
try (BufferedWriter writer = new BufferedWriter(new FileWriter(caminhoArquivo))) {
    for (int i = 1; i <= quantidade; i++) {
        int matricula = i;
        String nome = NOMES[random.nextInt(NOMES.length)] + " " +
        SOBRENOMES[random.nextInt(SOBRENOMES.length)];
        int nota = random.nextInt(11);

        writer.write(matricula + ";" + nome + ";" + nota);
        writer.newLine();
    }
}
```

A medição dos tempos de execução foi realizada com a classe `Benchmark`. Utilizamos `System.nanoTime()` para capturar o tempo imediatamente antes e após cada operação, isolando individualmente cada trecho medido. O benchmark foi executado **três vezes** de forma consecutiva dentro da mesma JVM, e os valores reportados nas tabelas correspondem à **média aritmética** das três execuções.

```
// Trecho do arquivo Benchmark.java
long inicioCarga = System.nanoTime();
// ... (leitura sequencial do arquivo e uso da função .adicionar())
long tempoCarga = System.nanoTime() - inicioCarga;

long t0 = System.nanoTime();
```

```
lista.pesquisar(ultimoAluno);
long tempoBuscaExistente = System.nanoTime() - t0;
```

Para mitigar o efeito de *cold start* e o overhead da compilação JIT (Just-In-Time), uma passagem silenciosa de aquecimento (*warm-up*) é executada sobre o menor arquivo antes do início das medições. Ainda assim, existem limitações metodológicas que devem ser reconhecidas: o **tempo de carga** inclui tanto as inserções na lista quanto a leitura sequencial do arquivo em disco, de modo que esse resultado reflete um custo misto de I/O e estrutura de dados — não o custo isolado de `adicionar`. Além disso, fatores como o *Garbage Collector* da JVM, o escalonamento de processos do sistema operacional e a variação de frequência do processador introduzem ruído que não é possível controlar neste experimento.

3.2 - Análise de complexidade

O benchmark foi executado sobre a `ListaEncadeada` utilizando os arquivos padronizados de 100.000, 200.000 e 400.000 registros. Cada cenário foi repetido **três vezes** e os valores abaixo representam a **média aritmética** dos resultados, expressos em milissegundos:

Operação / Tamanho da Lista	100.000 registros	200.000 registros	400.000 registros
Tempo de carga (I/O + Adicionar N elementos)	35,27 ms	51,46 ms	78,59 ms
Pesquisar (elemento existente no final)	3,32 ms	6,89 ms	14,81 ms
Pesquisar (elemento inexistente)	3,25 ms	6,97 ms	14,33 ms
Remover (último elemento)	2,94 ms	6,86 ms	14,13 ms
Quantidade de nós	~700 ns	~18 ns	~18 ns

Interpretação dos Dados

Os resultados obtidos são **consistentes** com as análises matemáticas apresentadas na seção anterior, com ressalvas experimentais descritas a seguir para cada operação:

1. **Tempo de carga — custo misto de I/O e inserção:** A operação `adicionar` individualmente possui complexidade O(1) e, portanto, popular uma lista com N elementos tem custo total O(N) em termos da estrutura. Os tempos crescem de ~35 ms (100K)

para ~78 ms (400K), mas esse fator de ~2,23× para uma entrada 4× maior é **inferior ao esperado para crescimento linear puro**. Isso indica que o custo de leitura do arquivo em disco — amortizado pelos buffers do sistema operacional e da JVM — domina o tempo total, ocultando o custo real da estrutura de dados. Esses resultados **não permitem validar diretamente** a complexidade de inserção, embora sejam compatíveis com o comportamento esperado de operações dominadas por I/O.

2. **Pesquisar e Remover — indicativo de crescimento linear (O(N)):** Estas operações percorrem a lista sequencialmente sem envolver leitura de arquivo, tornando-as os resultados mais representativos da complexidade da estrutura. Os tempos aproximadamente dobram ao dobrar N: a pesquisa de elemento existente sobe de 3,32 ms → 6,89 ms → 14,81 ms (razões de ~2,07× e ~2,15×), a pesquisa de inexistente de 3,25 ms → 6,97 ms → 14,33 ms (~2,14× e ~2,06×), e a remoção do último de 2,94 ms → 6,86 ms → 14,13 ms (~2,33× e ~2,06×). Esse padrão de duplicação é **consistente com crescimento linear O(N)**, embora variações pontuais reflitam ruído experimental.

3. **Quantidade de nós — compatível com O(1), sem confirmação empírica direta:** A operação lê o atributo `qtd` sem percorrer a lista. Os tempos medidos estão na ordem de dezenas a centenas de nanossegundos, o que **indica** ausência de crescimento proporcional ao tamanho da entrada. Contudo, uma única chamada por execução não é uma amostra estatisticamente suficiente nessa escala: para confirmar empiricamente O(1), seriam necessárias repetições em *loop* (e.g., milhares de chamadas) para obter uma média significativa. Os resultados são, portanto, **compatíveis** com O(1), mas não constituem evidência empírica conclusiva.

4 - Análise ArrayList e LinkedList

4.1 - LinkedList

Diferente de um Array o qual deve ter um tamanho definido de memória, a estrutura **LinkedList** do Java apresenta alocação dinâmica, alterando o espaço armazenado a medida que elementos são adicionados ou removidos. A **LinkedList** faz parte da **Java Collections** e implementa a *AbstractSequentialList*, o que - de forma prática - implica que o acesso a um elemento da lista é feita de forma sequencial, percorrendo um a um.

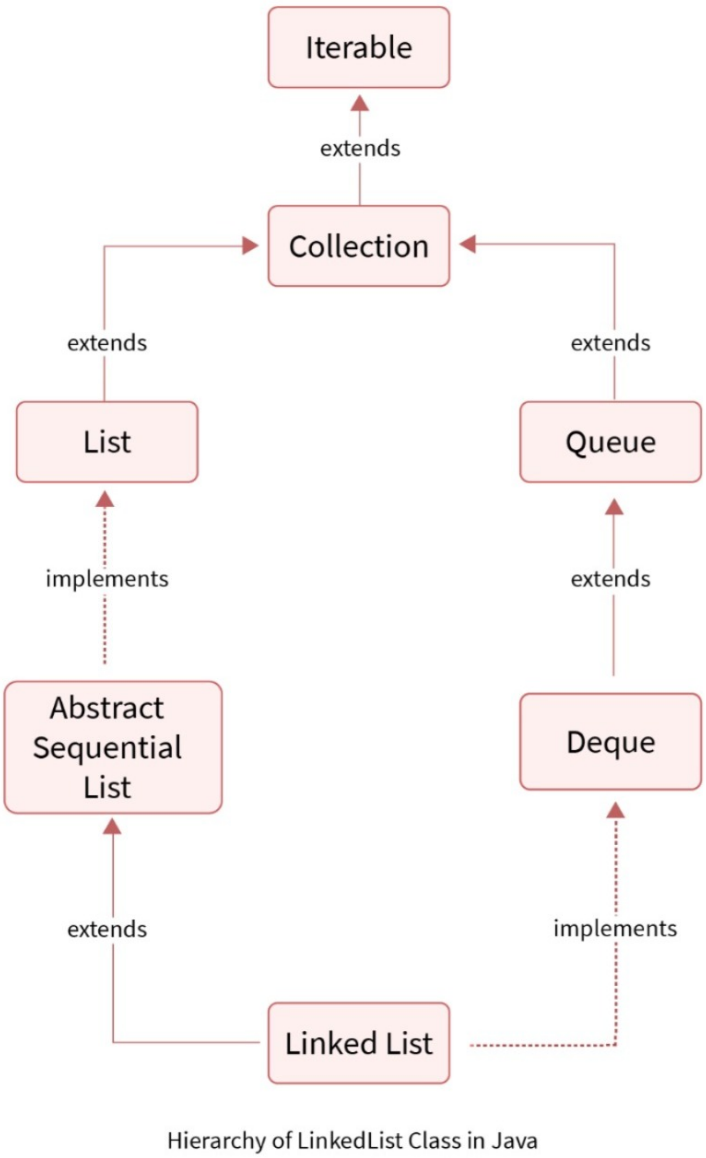

Figure 3: Implementação LinkedList

A estrutura mantém referências tanto para o primeiro quanto para o último nó da lista. Além disso, cada nó possui ponteiros para o elemento seguinte (next) e para o anterior (prev), caracterizando a estrutura como uma lista duplamente encadeada. Essa característica permite inserções eficientes nas extremidades, reduzindo o custo dessas operações para tempo constante.

```
// Implemtentação: apontando para primeiro e último elementos

public class LinkedList<E>
    extends AbstractSequentialList<E>
    implements List<E>, Deque<E>, Cloneable, java.io.Serializable
{
    transient int size = 0;

    /**
     * Pointer to first node.
     */
    transient Node<E> first;

    /**
     * Pointer to last node.
     */
    transient Node<E> last;
```

Métodos

Adicionar

A forma de se adicionar a um elemento em uma **LinkedList** é feito - principalmente - pelo método **add(E e)**. Além dele, também existem variações como **addFirst(E e)**, **addLast(E e)** e **add(int index, E element)**, cada uma com comportamentos distintos. Por existir ponteiros apontando para o primeiro e último elemento, as funções **addFirst** e **addLast** tem uma complexidade constante, por não serem alteradas de forma correlacionada a quantidade de elementos na lista.

O método **add(E e)** delega sua execução ao método **linkLast(E e)**, responsável por inserir o novo elemento ao final da lista utilizando a referência ao último nó. Como não há necessidade de percorrer a estrutura, a operação possui complexidade O(1).

```
// Método de adicionar elemento a lista encadeada
public boolean add(E e) {
    linkLast(e);
    return true;
}

// Método chamado por add
void linkLast(E e) {
    final Node<E> l = last;
```

```
    final Node<E> newNode = new Node<>(l, e, null);
    last = newNode;
    if (l == null)
        first = newNode;
    else
        l.next = newNode;
    size++;
    modCount++;
}
```

Contudo, o método **add(int index, E element)** - diferente dos citados anteriormente - acaba por ter complexidade Linear. Ao receber o índice como parâmetro, a lista precisa percorrer seus elementos nó a nó até alcançar a posição correspondente. Esse processo de localização possui custo linear em relação ao número de elementos armazenados.

O pior caso ocorre quando o índice está aproximadamente no meio da lista, pois - mesmo com a otimização que permite iniciar o percurso a partir do início ou do fim - ainda será necessário percorrer n/2 elementos. Assim, o tempo de execução cresce proporcionalmente ao tamanho da lista, caracterizando a complexidade **O(n)**.

Remover

A remoção de elementos em uma **LinkedList** pode ser realizada de duas formas principais: por índice ou valor sendo - para ambos os casos - complexidade **O(N)**.

Para a remoção por objeto, cada nó é comparado com o valor passado como parâmetro, em que toda a estrutura será percorrida no caso do elemento não estar presente, fazendo com que o tempo de execução cresça proporcionalmente ao número de elementos da lista.

A implementação também trata explicitamente o caso em que o valor buscado é nulo, decorrido o fato da **LinkedList** permitir a inserção de elementos **null**. Dessa forma, a verificação inicial evita a chamada do método equals sobre uma referência nula, o que resultaria em uma exceção.

```
public boolean remove(Object o) {
    if (o == null) {
        for (Node<E> x = first; x != null; x = x.next) {
            if (x.item == null) {
                unlink(x);
                return true;
            }
        }
    } else {
```

```
        for (Node<E> x = first; x != null; x = x.next) {
            if (o.equals(x.item)) {
                unlink(x);
                return true;
            }
        }
    }
    return false;
}
```

Já a remoção por índice pode ter um tempo de execução *menor* ao pior caso da remoção por objeto por percorrer apenas metade da lista, mas também cresce o tempo de execução de forma proporcional a quantidade de elementos na lista. caso o índice esteja na primeira metade da lista, a busca é realizada a partir do início (first); caso contrário, inicia-se a partir do final (last). Essa estratégia reduz a quantidade de elementos percorridos, limitando o percurso a - no máximo - *n/2* elementos. Apesar disso, sua complexidade se mantém **O(N)**.

```
// Remover por índice
public E remove(int index) {
    checkElementIndex(index);
    return unlink(node(index));
}

Node<E> node(int index) {
    // assert isElementIndex(index);

    if (index < (size >> 1)) {
        Node<E> x = first;
        for (int i = 0; i < index; i++)
            x = x.next;
        return x;
    } else {
        Node<E> x = last;
        for (int i = size - 1; i > index; i--)
            x = x.prev;
        return x;
    }
}
```

Busca

A busca em uma LinkedList pode ser realizada de duas formas principais: por índice ou por valor do elemento.

Na busca por objeto **contains(Object O)**, é utilizada internamente o método **indexOf(Object O)**, sendo parcialmente igual ao método de remoção por índice mas - ao invés de fazer uso do **unlink** e retornar um booleano, o método é responsável por retornar o índice no qual o objeto pertence. Por se tratar de um algoritmo que pecorrerá todos os elementos da lista em seu pior caso e crescer o tempo de execução dado o tamanho da lista, podemos dizer que o método é **O(N)**

```
public int indexOf(Object o) {
    int index = 0;
    if (o == null) {
        for (Node<E> x = first; x != null; x = x.next) {
            if (x.item == null)
                return index;
            index++;
        }
    } else {
        for (Node<E> x = first; x != null; x = x.next) {
            if (o.equals(x.item))
                return index;
            index++;
        }
    }
    return -1;
}
```

Na busca por índice, realizada pelo método **get(int index)**, o funcionamento é semelhante ao da remoção por índice no que diz respeito à localização do elemento. Em ambos os casos, é utilizado o método **node(index)**, responsável por percorrer a lista até alcançar a posição desejada. A principal diferença está na operação final em que, enquanto o método de remoção por meio do método unlink, o método get apenas acessa o valor armazenado no nó, por meio do atributo item, e o retorna.

```
public E get(int index) {
    checkElementIndex(index);
    return node(index).item;
}
```

4.2 - ArrayList

O **ArrayList** — assim como o **LinkedList** — faz parte do **Java Collections Framework**, sendo uma implementação baseada em array dinâmico.

Diferentemente de arrays tradicionais, o **ArrayList** possui redimensionamento automático, permitindo que sua capacidade aumente conforme novos elementos são adicionados.

Esse crescimento é realizado internamente por meio do método **grow()**, que é acionado quando a capacidade atual do array é insuficiente para armazenar novos elementos.

```
private Object[] grow(int minCapacity) {
    int oldCapacity = elementData.length;
    if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
        int newCapacity = ArraysSupport.newLength(oldCapacity,
                minCapacity - oldCapacity, /* minimum growth */
                oldCapacity >> 1           /* preferred growth */);
        return elementData = Arrays.copyOf(elementData, newCapacity);
    } else {
        return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
    }
}
```

Adicionar

Ao analisar o método **add(E e)** do **ArrayList**, estamos tratando de uma inserção no final da lista, ou seja, o elemento é adicionado diretamente na posição correspondente ao valor atual de size, seguida da sua incrementação. No cenário mais comum essa operação ocorre de forma direta no array interno, sem necessidade aumento do array e tendo tempo constante.

Entretanto, no pior caso, quando a capacidade do array já foi totalmente utilizada, é necessário redimensionar o array por meio do método **grow()**. Nesse processo, um novo array maior é criado e todos os elementos do array antigo são copiados para esse novo array.

```
private Object[] grow(int minCapacity) {
    int oldCapacity = elementData.length;
    if (oldCapacity > 0 || elementData != DEFAULTCAPACITY_EMPTY_ELEMENTDATA) {
        int newCapacity = ArraysSupport.newLength(oldCapacity,
                minCapacity - oldCapacity, /* minimum growth */
                oldCapacity >> 1           /* preferred growth */);
        return elementData = Arrays.copyOf(elementData, newCapacity);
    } else {
        return elementData = new Object[Math.max(DEFAULT_CAPACITY, minCapacity)];
```

```
}
}
```

Isso acaba por aumentar o tempo de execução dada a quantidade de elementos da lista, levando o método a ter complexidade linear (**(O(n))**).

Remover

Quando falamos dos métodos de remoção, temos uma remoção por índice e uma por objeto, ambas tendo complexidade linear. O método de remoção por índice **(remove(int index))** remove diretamente o elemento é acessado a partir do índice informado em tempo constante, mas - ao chamar o **fastRemove** - o método **System.arraycopy** é chamado, o qual acaba por deslocar todos os elementos à direita do índice removido uma posição para a esquerda, não deixando lacunas. Por conta disso, ao analisar o pior cenário - que seria remover o primeiro elemento e ter deslocar os elementos a da esquerda, percorrendo um a um - sua complexidade aumenta dada a quantidade de elementos na lista para o pior caso, sendo considerada **O(n)**.-

```
// Remoção por índice
public E remove(int index) {
    Objects.checkIndex(index, size);
    final Object[] es = elementData;

    @SuppressWarnings("unchecked") E oldValue = (E) es[index];
    fastRemove(es, index);

    return oldValue;
}

// Método chamado na remoção por índice
private void fastRemove(Object[] es, int i) {
    modCount++;
    final int newSize;
    if ((newSize = size - 1) > i)
        System.arraycopy(es, i + 1, es, i, newSize - i);
    es[size = newSize] = null;
}
```

Já na remoção por elemento, o método percorre elemento a elemento até encontrar o valor passado como parâmetro, o que, no pior caso, quando o elemento está na última posição da lista, exige percorrer todos os elementos, resultando em complexidade **O(n)** proporcional à quantidade de elementos armazenados. Contudo, mesmo no caso em que o elemento está

no início da lista, após a remoção o método **System.arraycopy** é chamado, assim como na remoção por índice, o que provoca o deslocamento de todos os elementos à direita uma posição para a esquerda. Dessa forma, também há um custo linear nesse cenário, mantendo a complexidade **O(n)**.

```
public boolean remove(Object o) {
    final Object[] es = elementData;
    final int size = this.size;
    int i = 0;
    found: {
        if (o == null) {
            for (; i < size; i++)
                if (es[i] == null)
                    break found;
        } else {
            for (; i < size; i++)
                if (o.equals(es[i]))
                    break found;
        }
        return false;
    }
    fastRemove(es, i);
    return true;
}
```

Busca

Os métodos de busca para o **ArrayList** podem ser por índice - sendo esse o de melhor desemepenho - e por objeto, sendo pior para o pior caso. Na busca por índice **get(int index)**, é passado um parâmetro para o método como parâmetro e o elemento é acessado pelo índice diretamente, levando o método a ser complexidade constante O(1).

```
public E get(int index) {
    Objects.checkIndex(index, size);
    checkForComodification();
    return root.elementData(offset + index);
}
```

Contudo, ao analisarmos a busca por elemento por meio do método **contains**, observamos que o método percorre elemento a elemento até encontrar o valor desejado ou até atingir o final da lista. No pior caso, quando o elemento não está presente ou se encontra na última posição, é necessário percorrer todo o array, fazendo com que o tempo de execução seja proporcional à

quantidade de elementos armazenados. Dessa forma, podemos caracterizar a operação como uma busca linear, com complexidade **O(n)**.

```
public boolean contains(Object o) {
    return indexOf(o) >= 0;
}

public int indexOf(Object o) {
    return indexOfRange(o, 0, size);
}

int indexOfRange(Object o, int start, int end) {
    Object[] es = elementData;
    if (o == null) {
        for (int i = start; i < end; i++) {
            if (es[i] == null) {
                return i;
            }
        }
    } else {
        for (int i = start; i < end; i++) {
            if (o.equals(es[i])) {
                return i;
            }
        }
    }
    return -1;
}
```

5 - Resultados e Comparação de Desempenho

O benchmark foi executado sobre as três implementações de `IColecao<Aluno>` — `ListaEncadeada` (customizada), `ListaEncadeadaArrayList` (wrapper de `java.util.ArrayList`) e `ListaEncadeadaLinkedList` (wrapper de `java.util.LinkedList`) — utilizando datasets de **400.000, 800.000 e 1.600.000 registros**. Esses tamanhos foram escolhidos para garantir que mesmo as operações mais rápidas (ArrayList) produzam tempos acima de ~5 ms, reduzindo a influência do overhead de medição.

Cada cenário foi repetido **três vezes** na mesma JVM (após aquecimento silencioso). Os valores reportados correspondem à **mediana** das três execuções — mais robusta que a média aritmética na presença de outliers causados por eventos esporádicos de GC ou recompilação JIT, comuns em sessões longas de benchmark sem isolamento de processo.

5.1 - Tabelas de Tempos de Execução

ListaEncadeada (implementação customizada)

Operação	400.000 registros	800.000 registros	1.600.000 registros
Tempo de carga (I/O + Adicionar)	213,46 ms	268,28 ms	563,72 ms
Pesquisar (existente — pior caso)	11,61 ms	30,84 ms	62,56 ms
Pesquisar (inexistente — varredura total)	9,97 ms	29,82 ms	62,88 ms
Remover (último elemento)	10,61 ms	29,32 ms	60,65 ms
Quantidade de nós	< 1 ms	< 1 ms	< 1 ms

ArrayList (java.util.ArrayList)

Operação	400.000 registros	800.000 registros	1.600.000 registros
Tempo de carga (I/O + Adicionar)	187,10 ms	—	—
Pesquisar (existente — pior caso)	7,10 ms	5,17 ms	15,33 ms
Pesquisar (inexistente — varredura total)	3,91 ms	5,23 ms	10,91 ms
Remover (último elemento)	6,83 ms	6,68 ms	11,46 ms
Quantidade de nós	< 1 ms	< 1 ms	< 1 ms

LinkedList (java.util.LinkedList)

Operação	400.000 registros	800.000 registros	1.600.000 registros
Tempo de carga (I/O + Adicionar)	145,69 ms	—	—
Pesquisar (existente — pior caso)	21,29 ms	33,44 ms	64,73 ms
Pesquisar (inexistente — varredura total)	19,02 ms	33,50 ms	64,71 ms
Remover (último elemento)	20,05 ms	31,16 ms	65,83 ms
Quantidade de nós	< 1 ms	< 1 ms	< 1 ms

Nota: as células de carga para 800K e 1,6M são omitidas da análise comparativa por serem metodologicamente não comparáveis entre implementações (ver seção 5.2).

5.2 - Interpretação e Aderência à Análise Matemática

Pesquisar e Remover nas listas encadeadas — consistente com O(N): Com os tamanhos maiores, o padrão de crescimento das listas encadeadas tornou-se mais claro. Na `ListaEncadeada` customizada, a pesquisa de elemento existente evolui de 11,61 ms → 30,84 ms → 62,56 ms ao dobrar N a cada passo (razões de ~2,66× e ~2,03×); na `java.util.LinkedList`, de 21,29 ms → 33,44 ms → 64,73 ms (~1,57× e ~1,94×). O padrão geral é **consistente com crescimento linear O(N)**: ao dobrar N, os tempos aproximadamente dobram, especialmente na transição de 800K para 1,6M. As razões afastadas de 2× na transição de 400K para 800K refletem ruído residual da JVM e possível efeito de recompilação JIT durante a primeira iteração da sessão.

Resultados do ArrayList — parcialmente estabilizados, ainda com variabilidade: Com entradas maiores, os tempos do `ArrayList` subiram para faixas mais mensuráveis: pesquisa de elemento existente atingiu 15,33 ms em 1,6M, e remoção do último ficou em 11,46 ms. Ainda assim, os dados de 400K e 800K mostram variabilidade considerável (ex: pesquisa de inexistente 3,91 ms → 5,23 ms → 10,91 ms, sem padrão de duplicação claro), indicando que as operações do `ArrayList` permanecem suscetíveis ao overhead de medição e às otimizações da JVM mesmo nessas escalas. **Os dados do ArrayList indicam tendência de crescimento linear, mas não permitem confirmar o padrão com o mesmo grau de confiança das listas encadeadas.** Um experimento rigoroso exigiria centenas de repetições com ferramenta especializada (ex.: JMH).

Tempo de carga — métrica não comparável entre estruturas: Conforme estabelecido na seção 3.2, o tempo de carga mistura custo de I/O com custo de inserção de forma inseparável.

Por isso, a comparação de tempos de carga entre implementações distintas **não mede diferença estrutural** e é omitida da análise comparativa.

Quantidade de nós — compatível com O(1): As três implementações retornam a contagem em tempo abaixo de 1 ms independente do tamanho, **compatível** com O(1), sem evidência empírica conclusiva por execução única.

5.3 - Comparação entre as Implementações

A tabela comparativa considera apenas as operações de busca e remoção, que isolam o comportamento estrutural sem custo de I/O. Os valores são da mediana em 1.600.000 registros, onde as diferenças são mais pronunciadas e estáveis:

Operação (1.600.000 registros)	Mais rápida	Intermediária	Mais lenta
Pesquisar (existente)	ArrayList (15,33 ms)	ListaEncadeada (62,56 ms)	LinkedList (64,73 ms)
Pesquisar (inexistente)	ArrayList (10,91 ms)	ListaEncadeada (62,88 ms)	LinkedList (64,71 ms)
Remover (último elemento)	ArrayList (11,46 ms)	ListaEncadeada (60,65 ms)	LinkedList (65,83 ms)

ArrayList foi consistentemente mais rápido nas operações de traversal, apesar de possuir a mesma complexidade assintótica O(N) das demais. Em 1,6M de registros, o `ArrayList` é entre **~4× e ~6× mais rápido** que as listas encadeadas em busca (15,33 ms vs. ~63–65 ms). Essa diferença expressiva de fator constante pode ser explicada pela **localidade de cache**: o `ArrayList` armazena referências em posições contíguas de memória, permitindo que a CPU antecipe e pré-carregue blocos adjacentes eficientemente (*cache prefetching*). As listas encadeadas alocam cada nó separadamente no heap, resultando em acessos dispersos e frequentes *cache misses* durante a traversal — custo proporcional ao tamanho da lista, e que explica o fator de diferença observado. Essa hipótese é consistente com os dados, mas não foi medida diretamente (nenhum profiler de cache foi utilizado).

A `ListaEncadeada` customizada e o `java.util.LinkedList` apresentam desempenho muito semelhante nas operações de busca e remoção em todas as escalas (ex.: 62,56 ms vs. 64,73 ms em pesquisa existente em 1,6M), o que é esperado: ambas percorrem nós alocados de forma dispersa no heap com o mesmo padrão de acesso não contíguo à memória. A hipótese de que o `java.util.LinkedList` poderia ser mais lento por alocar nós bidirecionais (ponteiro `prev` adicional) é **plausível mas não sustentada pelos dados** — as diferenças observadas estão dentro do ruído experimental.

O uso de `IColecao<T>` como interface comum demonstrou seu valor prático nesta comparação: a troca entre as três implementações exigiu apenas a alteração do `Supplier` no `Benchmark`, sem nenhuma modificação no código de medição — evidenciando o princípio de **programação para interfaces** e a facilidade de substituição que o polimorfismo proporciona.